

TEMA: hash para contraseña de usuarios**Resultados de Aprendizaje:**

- Que los alumnos comprendan los pasos necesarios para la realización de hash de contraseñas

DESARROLLO**Ejemplo :**

Para este ejemplo continuaremos a api comercial que comenzamos en las asiones anteriores y agregaremos un poco mas seguridad a los usuarios de modo tal que las contraseñas no se agregen de manera plana , si no que esten con un hash

Requerimientos

para poder realizar esta practica , debera de contar con:

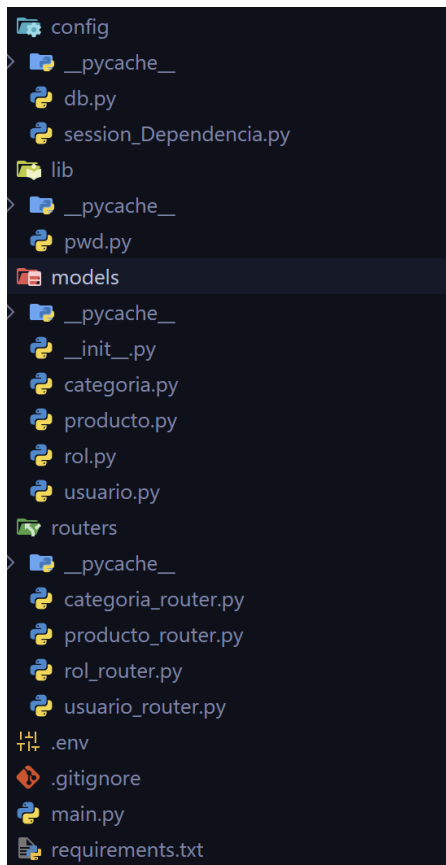
1. Un gestor de base de datos **mysql**
2. Crear una base de datos llamada **comercial**
3. Haber realizado la guias anteriores de la semana sobre el mantenimiento de la tablas incluidas rol y usuarios

Nota :

En esta guia de ejemplo realizaremos los pasos previos para la realizacion del CRUD de las tablas rol y usuario si ya las tiene puede dar una mirada breve del codigo

Desarrollo

1. Instale al proyecto las librerías **pwdlib** , con el comando **pip install pwdlib** , dicha librería es utilizada para proteger contraseñas mediante hashing.
2. Instale también el complemento **pip install pwdlib[argon2]**
3. Abra el proyecto creado en la practica anteriores
4. Crearemos los archivos **models/rol.py**, **models/usuario.py** , **routers/rol_router.py** , **routers/usuario_router.py** , **lib/pwd.py** al conjunto de archivos que ya tenemos creados con anterioridad



Creacion de modelos

5. Para comenzar crearemos el modelo de la tabla de rol, es probable que esto usted ya lo posee, y puede nada más verificar que posee los mismos códigos o similares, **models/rol.py**

```

1  from sqlmodel import SQLModel, Field
2  from datetime import datetime
3  from typing import Optional
4
5
6  class RolBase(SQLModel):
7      nombre: str = Field(nullable=False, max_length=255,
8      ..... min_length=3)
9      descripcion: Optional[str] = Field(default=None, max_length=255)
10
11
12  class Rol(RolBase, table=True):
13      ..... __tablename__ = "roles"
14      id: int | None = Field(default=None, primary_key=True)
15      created_at: datetime | None = Field(
16      ..... default_factory=datetime.utcnow)
17      updated_at: datetime | None = Field(
18      ..... default_factory=datetime.utcnow)
19
20
21  class RolCreate(RolBase):
22      ..... pass
23
24
25  class RolUpdate(RolBase):
26      ..... pass

```

6. Hoy crearemos los modelos para la tabla de usuarios del archivo **models/usuario.py**

```

1  from sqlmodel import SQLModel, Field
2  from datetime import datetime
3  from pydantic import EmailStr
4  from typing import Optional
5
6
7  class UsuarioBase(SQLModel):
8      ... username: str = Field(nullable=False, max_length=50,
9      ... ..min_length=3, unique=True)
10
11     ... password: str = Field(nullable=False, max_length=255,
12     ... ..min_length=3)
13     ... nombre: str = Field(nullable=False, max_length=255,
14     ... ..min_length=3)
15     ... apellido: str = Field(nullable=False, max_length=255,
16     ... ..min_length=3)
17     ... telefono: str = Field(nullable=False, max_length=9,
18     ... ..min_length=9)
19     ... correo: Optional[EmailStr] = Field(default=None, max_length=255)
20     ... id_rol: int = Field(nullable=False, foreign_key="roles.id")
21
22
23  class Usuario(UsuarioBase, table=True):
24      ... __tablename__ = "usuarios"
25      ... id: int | None = Field(default=None, primary_key=True)
26      ... created_at: datetime | None = Field(
27      ... ..default_factory=datetime.utcnow)
28      ... updated_at: datetime | None = Field(
29      ... ..default_factory=datetime.utcnow)
30
31

```

- Denote que la columna username tiene la etiqueta **unique=True** , esto permite que la base de datos haga unico cada nombre que es agregado a dicha columna
- Denote tambien que el correo es de tipo **EmailStr** , esta permite validar que el correo sea de tipo correo

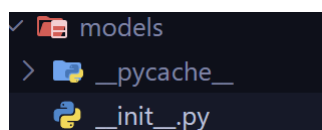
```

32 class UsuarioCreate(UsuarioBase):
33     ...pass
34
35
36 class UsuarioUpdate(UsuarioBase):
37     ...pass
38
39
40 class UsuarioUpdatePatch(SQLModel):
41     ...username: Optional[str] = Field(default=None, max_length=50,
42     ...min_length=3)
43
44     ...password: Optional[str] = Field(default=None, max_length=255,
45     ...min_length=3)
46     ...nombre: Optional[str] = Field(default=None, max_length=255,
47     ...min_length=3)
48     ...apellido: Optional[str] = Field(default=None, max_length=255,
49     ...min_length=3)
50     ...telefono: Optional[str] = Field(default=None, max_length=9,
51     ...min_length=9)
52     ...correo: Optional[EmailStr] = Field(default=None, max_length=255)
53     ...id_rol: Optional[int] = Field(default=None)
54

```

Impotacion de modelo

7. Importamos las clases que tienen la propiedad **table=True**, al archivo **models/__init__.py**, cada vez que agregue un nuevo modelo para crear la tabla en la base de datos deberá agregarla a este archivo



```

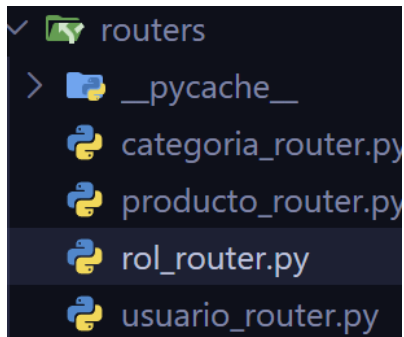
1 from models.categoria import Categoria
2 from models.producto import Producto
3 from models.rol import Rol
4 from models.usuario import Usuario

```

- ya puede correr la aplicacion con el comando **fastapi dev**
podra notar en la consola los comandos de creacion de las tablas en la base de datos .

Gestion de la api

- para poder gestionar la api y realizar el CRUD de rol desde la api modificaremos el archivo **routers/rol_router.py**



- realización de búsqueda de todos los roles

```
1  from datetime import datetime
2  from fastapi import APIRouter, HTTPException, status, Query
3  from models.rol import Rol, RolCreate, RolUpdate
4  from sqlmodel import select
5  from config.session_Dependencia import SessionDeDependencia
6
7
8  router = APIRouter()
9
10
11 @router.get("/roles", response_model=list[Rol], status_code=status.HTTP_200_OK)
12 def get_roles(session: SessionDeDependencia,
13               offset: int = Query(0, ge=0),
14               limit: int = Query(20, ge=1)):
15     ....
16     consulta = select(Rol).offset(offset).limit(
17     ....limit) # Limitar a 'limit' resultados por defecto
18     resultado_de_consulta = session.exec(consulta)
19     return resultado_de_consulta.all()
20     ....# return session.exec(select(Rol)).all()
```

11. buscar un rol dado el id

```
@router.get("/roles/{id}", response_model=Rol, status_code=status.HTTP_200_OK)
def get_rol(id: int, session: SessionDeDependencia):
    ... consulta = select(Rol).where(
    ...     Rol.id == id)
    ... resultado_de_consulta = session.exec(consulta).first()
    ... if not resultado_de_consulta:
    ...     raise HTTPException(status_code=404, detail="Rol no encontrado")
    ... return resultado_de_consulta
```

12. Para poder agregar un rol a la base de datos

```
@router.post("/roles", response_model=Rol, status_code=status.HTTP_201_CREATED)
def create_rol(datos_rol: RolCreate, session: SessionDeDependencia):
    ... rol_nuevo = Rol(
    ...     nombre=datos_rol.nombre, descripcion=datos_rol.descripcion)
    ... session.add(rol_nuevo)
    ... session.commit()
    ... session.refresh(rol_nuevo)
    ... return rol_nuevo
```

13. Eliminar el rol dado el id

```
@router.delete("/roles/{id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_rol(id: int, session: SessionDeDependencia):
    ... consulta = select(Rol).where(
    ...     Rol.id == id)
    ... resultado_de_consulta = session.exec(consulta).first()
    ... # verificación si existe un rol con ese id
    ... if not resultado_de_consulta:
    ...     raise HTTPException(status_code=404, detail="Rol no encontrado")
    ... # eliminación del rol
    ... session.delete(resultado_de_consulta)
    ... # ejecuta la consulta en la base de datos
    ... session.commit()
    ... return None
```

14. Para finalizar el CRUD , realizaremos la actualización de la tabla de roles

```
@router.put("/roles/{id}", response_model=Rol, status_code=status.HTTP_200_OK)
def update_rol(id: int, datos_rol: RolUpdate, session: SessionDeDependencia):
    ... consulta = select(Rol).where(Rol.id == id)
    ... resultado_de_consulta = session.exec(consulta).first()
    ... # verificación si existe un rol con ese id
    ... if not resultado_de_consulta:
    ...     ... raise HTTPException(status_code=404, detail="Rol no encontrado")
    ... # actualización de datos
    ... resultado_de_consulta.nombre = datos_rol.nombre
    ... # actualización de la descripción solo si se proporciona un valor
    ... if datos_rol.descripcion is not None:
    ...     ... resultado_de_consulta.descripcion = datos_rol.descripcion

    ... resultado_de_consulta.updated_at = datetime.utcnow()
    ... # agregar la consulta a la session
    ... session.add(resultado_de_consulta)
    ... # ejecuta la consulta en la base de datos
    ... session.commit()
    ... session.refresh(resultado_de_consulta)
    ... return resultado_de_consulta
```

Llama a router desde el punto de montaje de la app

15. Hemos creado todas las rutas pero aún no hemos podido probarlas , para poder realizar esto deberá de realizar la llamada de las rutas de roles desde el archivo **main.py** de la siguiente manera

```

1  from fastapi import FastAPI, status
2  from contextlib import asynccontextmanager
3  from config.db import crear_db_y_tablas
4  from routers.categoria_router import router as categorias_router
5  from routers.producto_router import router as productos_router
6  from routers.rol_router import router as rol_router
7
8
9  import models as models
10
11
12  @asynccontextmanager
13  async def lifespan(app: FastAPI):
14      ... crear_db_y_tablas()
15      ... yield
16
17
18  app = FastAPI(lifespan=lifespan)
19  app.title = "API Tienda la Cachaca"      "Cachaca": Unknown word.
20  app.version = "0.0.1"
21
22
23  @app.get("/", summary="Comprobando estado de api", status_code=status.HTTP_200_OK)
24  async def home():
25      ... return {"message": "ok"}
26
27  # el parámetro tags sirve para agrupar las rutas en la documentation de fastapi
28  app.include_router(categorias_router, tags=["categorias"])
29  app.include_router(productos_router, tags=["productos"])
30  app.include_router(rol_router, tags=["roles"])
31

```

16. Hoy es el momento para que usted pueda realizar todas las pruebas correspondientes a la table de roles

Manejamiento de la tabla de usuarios

Aun no hemos terminado viene la parte mas interesante y es el CRUD de la tabla de usuarios **demodo que podamos realizar un hash para las contraseñas**

Funciones para generar y verificar hash

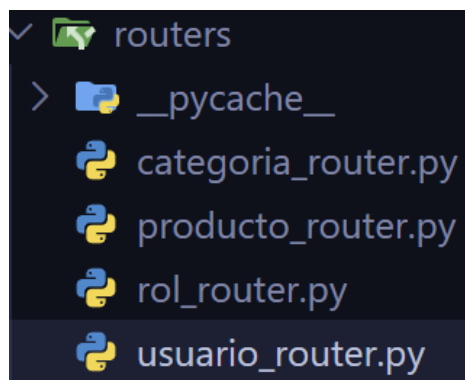
Para poder **generar los hash y verificar estos hash**, modificaremos el archivo **lib/pwd.py** de la siguiente manera

```
1  from pwdlib import PasswordHash
2
3  password_hash = PasswordHash.recommended()
4
5
6  def verify_password(plain_password, hashed_password):
7      ...return password_hash.verify(plain_password, hashed_password)
8
9
10 def get_password_hash(password):
11     ...return password_hash.hash(password)
```

- La funcion `verify_password` recibe dos parametros la contraseña plana y la contraseña en formato hash y nos indicara si dichos elementos coinciden o no

Gestion de la api para usuarios

1. para poder gestionar la api y realizar el CRUD de usuarios modificaremos el archivo **routers/usuario_router.py**



2. realizacion de busqueda de todos los usuarios

```
1  from fastapi import APIRouter, HTTPException, status, Query
2  from sqlmodel import select
3  from config.session_Dependencia import SessionDeDependencia
4  from models.rol import Rol
5  from models.usuario import Usuario, UsuarioCreate, UsuarioUpdate, UsuarioUpdatePatch
6  from datetime import datetime
7  from lib.pwd import get_password_hash
8  router = APIRouter()
9
10
11 @router.get("/usuarios", response_model=list[Usuario], status_code=status.HTTP_200_OK)
12 async def get_usuarios(session: SessionDeDependencia,
13     .....: offset: int = Query(
14     .....:     0, ge=0),
15     .....: limit: int = Query(
16     .....:     100, ge=1,
17     .....: )):
18     .....: consulta = select(Usuario).offset(offset).limit(limit)
19     .....: resultado_de_consulta = session.exec(consulta)
20     .....: return resultado_de_consulta.all()
```

3. buscar un usuario dado el id

```
@router.get("/usuarios/{id}", response_model=Usuario, status_code=status.HTTP_200_OK)
async def get_usuario(id: int, session: SessionDeDependencia):
    .....: consulta = select(Usuario).where(Usuario.id == id)
    .....: resultado_de_consulta = session.exec(consulta).first()
    .....: if not resultado_de_consulta:
    .....:     .....: raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
    .....:     .....: detail="Usuario no encontrado")
    .....: return resultado_de_consulta
```

4. Para poder agregar un usuario a la base de datos , no olvide que para la creacion del usuario debera de realizar el hash de la contraseña.

```

@router.post('/usuarios', response_model=Usuario, status_code=status.HTTP_201_CREATED)
async def create_usuario(datos_usuario: UsuarioCreate, session: SessionDeDependencia):

    .... usuario_nuevo = Usuario(
    ....     password=get_password_hash(datos_usuario.password),
    ....     nombre=datos_usuario.nombre,
    ....     apellido=datos_usuario.apellido,
    ....     telefono=datos_usuario.telefono,
    .... )

    .... consulta = select(Usuario).where(
    ....     Usuario.username == datos_usuario.username)
    .... usuario_existente = session.exec(consulta).first()
    .... if usuario_existente:
    ....     raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
    ....         detail="El nombre de usuario ya está en uso")

    .... usuario_nuevo.username = datos_usuario.username
    .... if datos_usuario.correo:
    ....     usuario_nuevo.correo = datos_usuario.correo

    .... if datos_usuario.id_rol:
    ....     consulta = select(Rol).where(Rol.id == datos_usuario.id_rol)
    ....     rol = session.exec(consulta).first()
    ....     if not rol:
    ....         raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
    ....             detail="Rol no encontrado")

    .... usuario_nuevo.id_rol = datos_usuario.id_rol
    .... session.add(usuario_nuevo)
    .... session.commit()
    .... session.refresh(usuario_nuevo)
    .... return usuario_nuevo

```

5. Eliminar el usuario dado el id

```

@router.delete('/usuarios/{id}', status_code=status.HTTP_204_NO_CONTENT)
async def delete_usuario(id: int, session: SessionDeDependencia):
    .... consulta = select(Usuario).where(Usuario.id == id)
    .... resultado_de_consulta = session.exec(consulta).first()
    .... if not resultado_de_consulta:
    ....     raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
    ....         detail="Usuario no encontrado")
    .... session.delete(resultado_de_consulta)
    .... session.commit()
    .... return None

```

6. Para finalizar el CRUD , realizaremos la actualización de la tabla de usuarios

```
@router.put('/usuarios/{id}', response_model=Usuario, status_code=status.HTTP_200_OK)
async def update_usuario(id: int, datos_usuario: UsuarioUpdate, session: SessionDeDependencia):
    consulta = select(Usuario).where(Usuario.id == id)
    usuario = session.exec(consulta).first()

    if not usuario:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
                             detail='Usuario no encontrado')

    if datos_usuario.id_rol:
        consulta = select(Rol).where(Rol.id == datos_usuario.id_rol)
        rol = session.exec(consulta).first()
        if not rol:
            raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
                                 detail="Rol no encontrado")

    if datos_usuario.correo:
        usuario.correo = datos_usuario.correo

    usuario.username = datos_usuario.username
    usuario.nombre = datos_usuario.nombre
    usuario.apellido = datos_usuario.apellido
    usuario.telefono = datos_usuario.telefono
    usuario.id_rol = datos_usuario.id_rol
    usuario.updated_at = datetime.utcnow()

    session.add(usuario)
    session.commit()
    session.refresh(usuario)
    return usuario
```

7. Como boro el actualizar utilizando el verbo patch

```
@router.patch('/usuarios/{id}', response_model=Usuario, status_code=status.HTTP_200_OK)
async def patch_usuario(id: int, datos_usuario: UsuarioUpdatePatch, session: SessionDeDependencia):
    consulta = select(Usuario).where(Usuario.id == id)
    usuario = session.exec(consulta).first()

    if not usuario:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
                             detail='Usuario no encontrado')

    if datos_usuario.correo:
        usuario.correo = datos_usuario.correo

    if datos_usuario.id_rol:
        consulta = select(Rol).where(Rol.id == datos_usuario.id_rol)
        rol = session.exec(consulta).first()
        if not rol:
            raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
                                 detail="Rol no encontrado")
        usuario.id_rol = datos_usuario.id_rol

    usuario.sqlmodel_update(datos_usuario.model_dump(exclude_unset=True))
    usuario.updated_at = datetime.utcnow()

    session.add(usuario)
    session.commit()
    session.refresh(usuario)
    return usuario
```

Llama a router desde el punto de montaje de la app

8. Hemos creado todas las rutas pero aun no hemos podido probarlas, para poder realizar esto debera de realizar la llamada de las rutas de usuarios desde el archivo **main.py** de la siguiente manera

```
1  from fastapi import FastAPI, status
2  from contextlib import asynccontextmanager
3  from config.db import crear_db_y_tablas
4  from routers.categoria_router import router as categorias_router
5  from routers.producto_router import router as productos_router
6  from routers.rol_router import router as rol_router
7  from routers.usuario_router import router as usuario_router
8
9  import models as models
10
11
12  @asynccontextmanager
13  async def lifespan(app: FastAPI):
14      ... crear_db_y_tablas()
15      ... yield
16
17
18  app = FastAPI(lifespan=lifespan)
19  app.title = "API Tienda la Cachaca"    "Cachaca": Unknown word.
20  app.version = "0.0.1"
21
22
23  @app.get("/", summary="Comprobando estado de api", status_code=status.HTTP_200_OK)
24  async def home():
25      ... return {"message": "ok"}
26
27  # el parámetro tags sirve para agrupar las rutas en la documentation de fastapi
28  app.include_router(categorias_router, tags=["categorias"])
29  app.include_router(productos_router, tags=["productos"])
30  app.include_router(rol_router, tags=["roles"])
31  app.include_router(usuario_router, tags=["usuarios"])
```

9. Hoy es el momento para que usted pueda realizar todas las pruebas correspondientes



EJERCICIOS DE PRÁCTICA:

- Realice los cambio correspondientes en su proyecto para validar que los username de los usaurios sean unicos
- Realice los cambios del proyecto para realizar los has de las contraseñas cuando se agregue un nuevo usuario